

PocketBooks Google OAuth Documentation (Phase 1)

Project Title

PocketBooks: Student Expense Tracker (Web)

Objective

Implement Google Sign-In as an alternative login method and integrate it with the existing authentication flow.

Implemented Features

- Google Sign-In button on the login page using `@react-oauth/google`
- Frontend obtains Google credential token and sends it to backend
- Backend endpoint `POST /api/google-login` handles Google-based login
- Auto-registration for first-time Google users
- Existing users can be linked with Google account (`googleId`, `authProvider`)
- Session is stored in `localStorage` and user is redirected to dashboard

Files Updated

- `frontend/src/pages/AuthPage.jsx`
- `frontend/server/index.js`
- `frontend/server/db.json`
- `frontend/package.json`
- `frontend/.env`

Frontend Integration Details

AuthPage

- Reads Google Client ID from environment:

```
```jsx
const GOOGLE_CLIENT_ID = import.meta.env.VITE_GOOGLE_CLIENT_ID || ""
```
```

- Uses `GoogleOAuthProvider` only when client ID is configured
- Renders `GoogleLogin` button in Sign In mode
- On success, sends token to backend:

```
```js
await fetch('/api/google-login', {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ token: credentialResponse.credential }),
})
```
```

Backend Integration Details

Endpoint

``POST /api/google-login``

Logic

1. Validate token exists
2. Decode JWT payload (demo mode)
3. Extract email/name/sub from token
4. Find user by normalized email
5. If not found, create user with:
 - ``googleId``
 - ``authProvider: "google"``
6. Return app session token and sanitized user object

Database Schema Changes

User entries now support:

- ``googleId`` (string)
- ``authProvider`` ("google" or omitted/other for classic login)

Example record:

```
```json
{
 "id": "dc0bc490-fabd-44de-b6b7-88a1f6a53806",
 "name": "Jascha Base",
 "email": "basejascha@gmail.com",
 "googleId": "105428528925124386605",
 "authProvider": "google",
 "createdAt": "2026-03-11T11:45:31.282Z"
}
```
```

Environment Setup

In ``frontend/.env``:

```
```env
VITE_GOOGLE_CLIENT_ID=YOUR_GOOGLE_WEB_CLIENT_ID.921021612299-1okgsapcvr
8v5656slpsvuiibvds6q8q.apps.googleusercontent.com
```
```

Google Cloud Console Setup

1. Open **Google Cloud Console**
2. Select/create project
3. Configure **OAuth consent screen**(External)
4. Go to **APIs & Services > Credentials**

5. Create **OAuth client ID**
6. Choose **Web application (FOR NOW)**
7. Add Authorized JavaScript origins:
 - `http://localhost:5174`
 - `http://127.0.0.1:5174`
8. Copy Client ID to `frontend/.env`

Run Instructions

From `frontend` folder:

```
``bash
npm install
npm run dev:api
npm run dev
``
```

- Frontend: `http://localhost:5174`
- API: `http://localhost:4000`

Common Error + Fix

`Error 401: invalid\_client`

Causes:

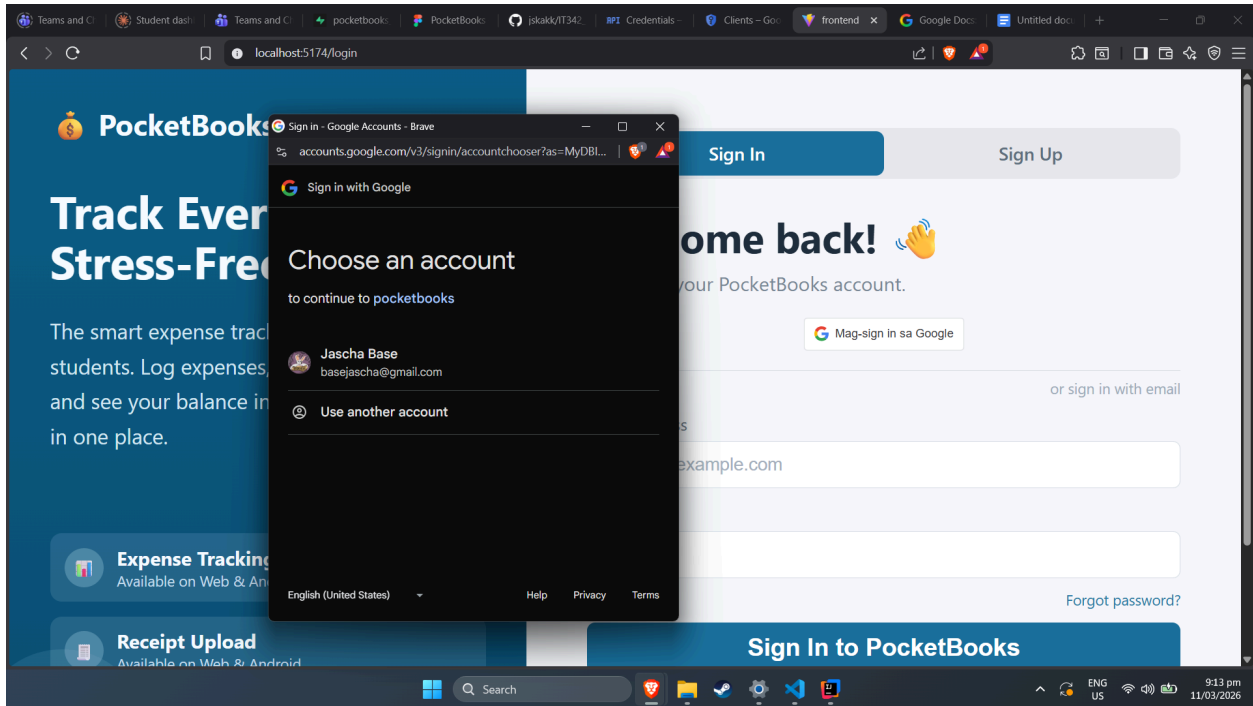
- Wrong/placeholder client ID in `.env`
- OAuth client is not **Web application**
- Missing `http://localhost:5173` in Authorized JavaScript origins
- Frontend not restarted after `.env` change

Fix:

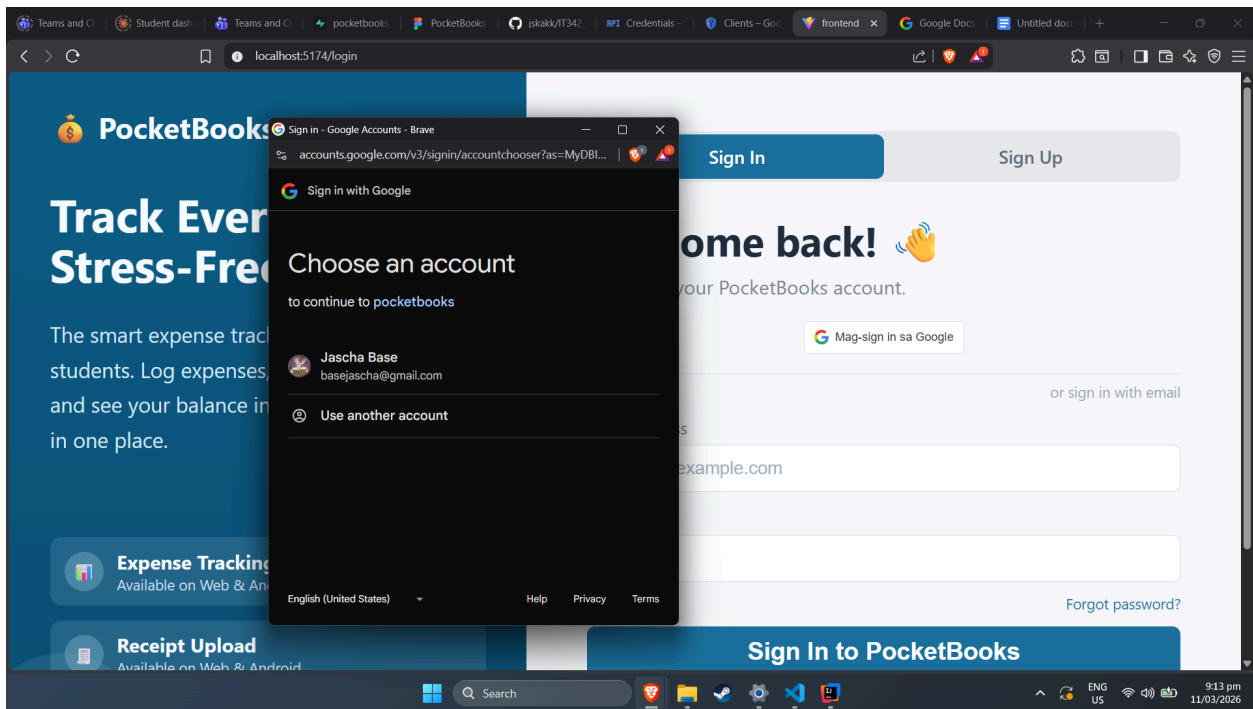
- Use the real Web Client ID
- Verify origins in Google Cloud
- Restart `npm run dev`

Screenshots Required for Submission

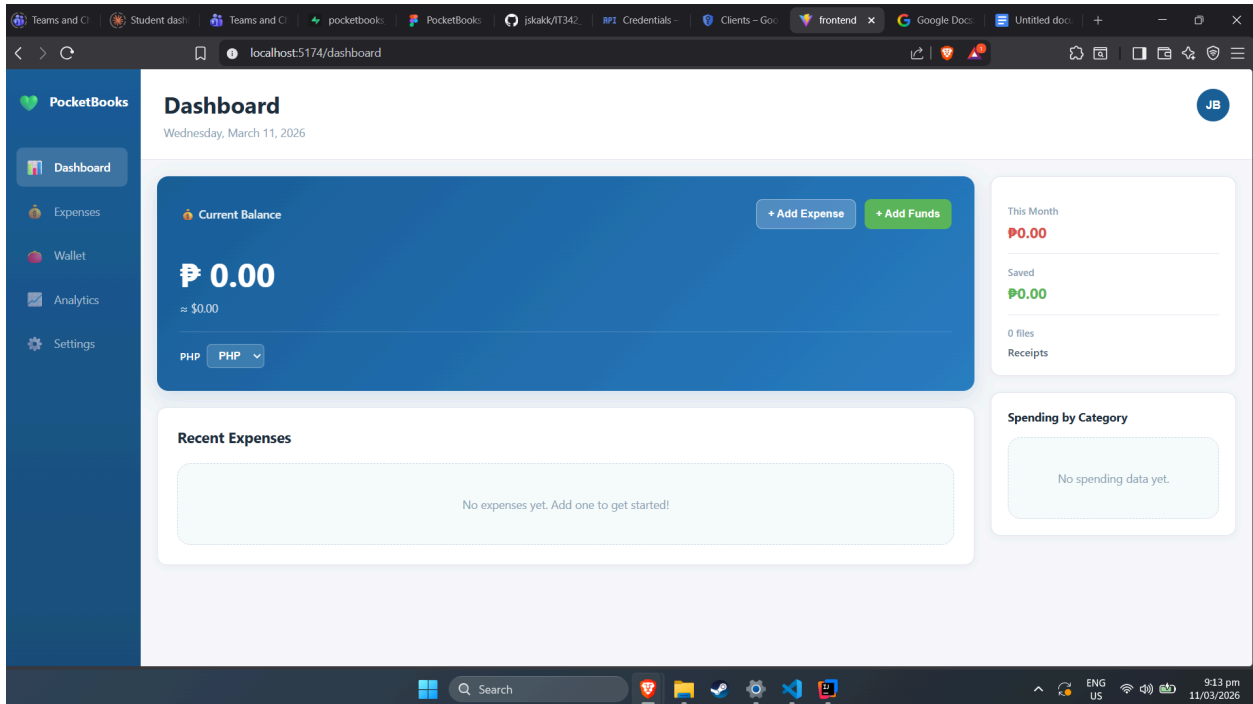
1. Login page with Google Sign-In button



2. Google account chooser popup



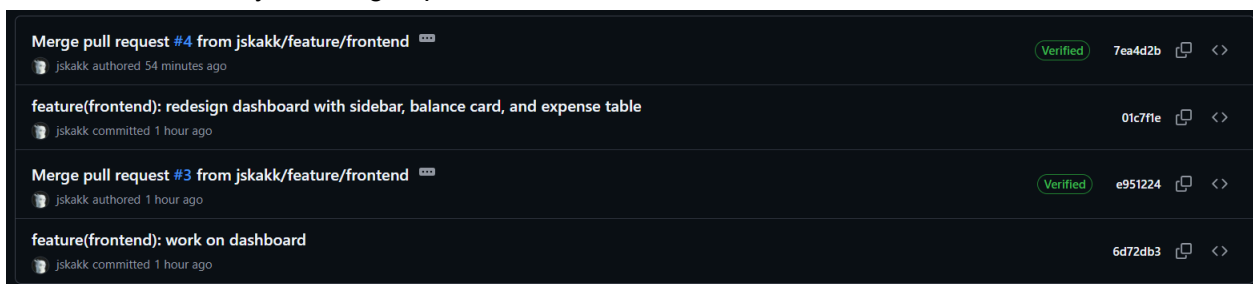
3. Successful redirect to dashboard after Google login



4. `frontend/server/db.json` showing user with `googleId` and `authProvider`

```
{
  "id": "dc0bc490-fabd-44de-b6b7-88a1f6a53806",
  "name": "Jascha Base",
  "email": "basejascha@gmail.com",
  "passwordHash": "$2b$12$tKovulyGYKCDMQKfkgJLp.jmVgTwART1aYG46VcwRfY4oepnVftz0",
  "createdAt": "2026-03-11T11:45:31.282Z",
  "googleId": "105428528925124386605",
  "authProvider": "google"
},
{
  "id": "google-user-demo-001",
  "name": "Demo Google User",
  "email": "demo.google@example.com",
  "googleId": "110169234559156813851",
  "authProvider": "google",
  "createdAt": "2026-03-11T12:00:00.000Z"
}
],
```

5. Git commit history showing implementation commits



Current Status

- Google Sign-In integration: Completed
- Backend endpoint integration: Completed
- Database integration: Completed
- Documentation: Completed
- Screenshot capture: In progress